ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC SƯ PHẠM

BÁO CÁO TIỂU LUẬN

MÔN HỌC: HỌC MÁY NÂNG CAO

Giáo viên hướng dẫn : TS. Phan Trần Đức Minh

Sinh viên thực hiện : Lê Thanh Thảo

MSSV : 3110524028

Lớp : 24CKDL

Đà Nẵng, ngày 21 tháng 9 năm 2026

Mục lục

DANH MỤC CÁC TỪ VIẾT TẮT **5**

LỜI CẢM ƠN **6**

LỜI MỞ ĐẦU **7**

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN **8**

1.1. Bối cảnh 8

1.2. Mục tiêu của đề tài 8

1.3. Phạm vi của báo cáo 9

1.4. Kiến trúc tổng quát của hệ thống 9

CHƯƠNG 2: MÔ HÌNH SVM - CƠ SỞ LÝ THUYẾT **10**

2.1. Khái niệm Support Vector Machine 10

2.2. Ý tưởng Maximum Margin 10

2.3. Soft Margin và tham số C 11

2.4. Hàm mất mát Hinge 11

2.5. Dạng đối ngẫu và Support Vector 11

2.6. Kernel Trick 12

2.7. Bốn Kernel được sử dụng trong đề tài 12

2.7.1. Kernel Linear 12

2.7.2. Kernel RBF 12

2.7.3. Kernel Polynomial 12

2.7.4. Kernel Sigmoid 12

2.8. SVM đa lớp 13

2.9. Các chỉ số đánh giá 13

CHƯƠNG 3: DỮ LIỆU VÀ TRỰC QUAN HÓA DỮ LIỆU **14**

3.1. Giới thiệu bộ dữ liệu Iris 14

3.2. Phân bố số lượng mẫu 14

3.3. Thống kê mô tả 15

3.4. Nhận xét về khả năng phân tách 15

3.5. Trực quan hóa hai chiều 16

CHƯƠNG 4: HUẤN LUYỆN MÔ HÌNH SVM: QUY TRÌNH VÀ KIỂM TRA TÍNH HIỆU QUẢ 17

4.1. Quy trình huấn luyện 17

4.2. Cấu hình bốn mô hình 17

4.3. Kết quả đánh giá thực tế 18

4.4. Biểu đồ so sánh Accuracy 18

4.5. Confusion Matrix 19

4.6. Support Vector và thời gian thực thi 20

4.7. Kiểm tra tính hiệu quả 20

4.8. Nhận xét kết quả thực nghiệm 20

CHƯƠNG 5: CÁC CÔNG NGHỆ SỬ DỤNG ĐỂ TRIỂN KHAI TRỰC TUYẾN . 22

5.1. Python và scikit-learn 22

5.2. Joblib 22

5.3. FastAPI 22

5.4. Pydantic 22

5.5. React và Vite 23

5.6. Node.js và Express 23

5.7. Supabase 23

5.8. Render 23

5.9. Kiến trúc triển khai thực tế 23

CHƯƠNG 6: CHI TIẾT QUY TRÌNH TRIỂN KHAI TRỰC TUYẾN VÀ VẬN HÀNH 24

Giai đoạn 1 – Huấn luyện và tạo artifact 24

Giai đoạn 2 – Kiểm tra API cục bộ 24

Giai đoạn 3 – Kiểm tra Health Check 25

Giai đoạn 4 – Đóng gói frontend 25

Giai đoạn 5 – Đẩy repository và triển khai Render 25

Giai đoạn 6 – Vận hành sau triển khai 26

Luồng một yêu cầu dự đoán 26

Lưu ý về bảo mật và vận hành 26

CHƯƠNG 7: DEMO VÀ HƯỚNG DẪN SỬ DỤNG WEB CHO NGƯỜI DÙNG . . 27

7.1. Tổng quan quy trình truy cập và Giao diện chính 27

7.1.1. Chức năng 1 – Nhận diện & Huấn luyện SVM 30

7.1.2. Chức năng 2 – Đoán hoa 33

7.1.3. Chức năng 3 – Phân tích dữ liệu 34

7.1.4. Chức năng 4 – Model Benchmark 36

7.1.5. Chức năng 5 – Lịch sử nhận diện 36

7.1.6. Chức năng 6 – Lịch sử thí nghiệm toàn hệ thống (Dành cho quyền quản trị viên) 37

7.1.7. Chức năng 7 – Thống kê hoạt động toàn hệ thống (Dành cho quyền quản trị viên) 37

7.2. Quy trình sử dụng đề xuất cho người mới 38

KẾT LUẬN 39

PHỤ LỤC 40

TÀI LIỆU THAM KHẢO 41

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh nguyên bản	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
ML	Machine Learning	Học máy
SVM	Support Vector Machine	Máy vector hỗ trợ
SVC	C-Support Vector Classification	Phân loại bằng máy vector hỗ trợ
CSV	Comma-Separated Values	Giá trị phân tách bằng dấu phẩy
OvO	One-vs-One	Chiến lược phân loại một-với-một
API	Application Programming Interface	Giao diện lập trình ứng dụng
ASGI	Asynchronous Server Gateway Interface	Giao diện cổng máy chủ bất đồng bộ
REST	Representational State Transfer	Kiểu kiến trúc truyền trạng thái biểu diễn
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
JSON	JavaScript Object Notation	Định dạng dữ liệu JavaScript Object Notation
CORS	Cross-Origin Resource Sharing	Chia sẻ tài nguyên giữa các nguồn khác nhau
DOM	Document Object Model	Mô hình đối tượng tài liệu
UI/UX	User Interface / User Experience	Giao diện và trải nghiệm người dùng
SPA	Single Page Application	Ứng dụng một trang
URL	Uniform Resource Locator	Địa chỉ tài nguyên đồng nhất
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
PaaS	Platform as a Service	Nền tảng dưới dạng dịch vụ

LỜI CẢM ƠN

Để hoàn thành đề tài **"Hệ thống nhận diện loài hoa Iris bằng mô hình Support Vector Machine (SVM) và Ứng dụng Web trực tuyến"** một cách trọn vẹn nhất, bên cạnh sự nỗ lực của bản thân, em đã nhận được sự hỗ trợ, giúp đỡ rất nhiệt tình từ Thầy Phan Trần Đức Minh và bạn bè.

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Giảng viên hướng dẫn — Thầy Phan Trần Đức Minh, người đã tận tình chỉ bảo, định hướng tư duy khoa học và đưa ra những lời khuyên chuyên môn vô cùng quý báu trong suốt quá trình em thực hiện. Những tri thức và sự tận tụy của Thầy không chỉ giúp em hoàn thiện sản phẩm này mà còn là hành trang quý giá cho con đường học tập của em sau này.

Em xin trân trọng cảm ơn Ban Giám hiệu Trường Đại học Sư phạm - Đại học Đà Nẵng đã đưa môn **Học máy nâng cao** vào chương trình đào tạo. Môn học thực sự bổ ích, giúp em tiếp cận với những công nghệ hiện đại, phát triển tư duy logic và mở ra nhiều cơ hội ứng dụng thực tiễn trong ngành Khoa học dữ liệu hiện nay.

Đồng thời, em xin gửi lời cảm ơn chân thành đến quý Thầy/Cô trong Khoa Toán-Tin đã truyền đạt cho em những nền tảng kiến thức chuyên môn vững chắc và tạo điều kiện học tập tốt nhất cho em trong suốt thời gian qua.

Dù đã cố gắng đầu tư thời gian và tâm huyết để hoàn thiện báo cáo cũng như xây dựng sản phẩm Web một cách chỉn chu nhất, song do hạn chế về mặt kinh nghiệm thực tế nên đề tài khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp, nhận xét và chỉ bảo quý báu từ Thầy để sản phẩm của em ngày càng được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Đà Nẵng, ngày 21 tháng 9 năm 2026

Sinh viên thực hiện

Lê Thanh Thảo

LỜI MỞ ĐẦU

Trong những năm gần đây, trí tuệ nhân tạo và học máy ngày càng được ứng dụng rộng rãi trong các bài toán nhận dạng, phân loại và hỗ trợ ra quyết định. Trong số các thuật toán học có giám sát, Support Vector Machine (SVM) là một phương pháp có cơ sở lý thuyết chặt chẽ, đặc biệt phù hợp với các bài toán phân loại có số lượng mẫu vừa phải và không gian đặc trưng có khả năng phân tách tốt.

Bài toán phân loại hoa *Iris* là một bài toán kinh điển trong học máy. Bộ dữ liệu gồm 150 mẫu thuộc ba loài *Iris setosa*, *Iris versicolor* và *Iris virginica*; mỗi mẫu được mô tả bởi bốn đặc trưng: chiều dài đài hoa, chiều rộng đài hoa, chiều dài cánh hoa và chiều rộng cánh hoa. Do cấu trúc dữ liệu đơn giản nhưng vẫn thể hiện cả trường hợp phân tách tuyến tính và phi tuyến, bộ dữ liệu này rất thích hợp để minh họa nguyên lý hoạt động của SVM và kỹ thuật Kernel.

Mục tiêu của đề tài là xây dựng một hệ thống hoàn chỉnh từ khâu chuẩn bị dữ liệu, huấn luyện mô hình, đánh giá hiệu năng đến triển khai trực tuyến. Điểm nổi bật của hệ thống là huấn luyện và lưu đồng thời bốn mô hình SVM sử dụng các Kernel Linear, RBF, Polynomial và Sigmoid. Kết quả huấn luyện được lưu thành các tệp mô hình và tệp thống kê để phục vụ API. Trên lớp giao diện web, người dùng có thể nhập thông số một mẫu hoa, thực hiện dự đoán, xem trực quan hóa dữ liệu, điều chỉnh siêu tham số trong SVM Playground, thực hiện thí nghiệm, phân tích tệp dữ liệu và xem lịch sử dự đoán.

Bài báo cáo trình bày từ nền tảng toán học của SVM, đặc điểm bộ dữ liệu, quy trình huấn luyện và kiểm tra, kiến trúc phần mềm, công nghệ triển khai trực tuyến cho đến hướng dẫn sử dụng hệ thống. Các số liệu đánh giá trong báo cáo được lấy từ các tệp kết quả thực tế của dự án, đặc biệt là `metrics.json` và `weights.json`, nhằm bảo đảm tính nhất quán giữa báo cáo và chương trình đã xây dựng.

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

1.1. Bối cảnh

Trong một hệ thống học máy thực tế, việc xây dựng một mô hình phân loại chỉ là một phần của bài toán. Một hệ thống có giá trị sử dụng cần trải qua chuỗi công việc gồm thu thập dữ liệu, tiền xử lý, huấn luyện, đánh giá, đóng gói mô hình, xây dựng API, xây dựng giao diện và triển khai lên môi trường trực tuyến. Vì vậy, đề tài được thực hiện theo hướng xây dựng một sản phẩm hoàn chỉnh thay vì chỉ dừng ở việc chạy một đoạn mã huấn luyện.

Bộ dữ liệu *Iris* được lựa chọn vì có cấu trúc rõ ràng, số lượng mẫu vừa phải và ba lớp tương đối dễ phân biệt. Đặc biệt, các đặc trưng về cánh hoa có khả năng phân tách các loài khá rõ, trong khi ranh giới giữa *versicolor* và *virginica* vẫn tạo ra tình huống có tính thử thách cho mô hình. Đây là điều kiện phù hợp để khảo sát các Kernel khác nhau của SVM.

1.2. Mục tiêu của đề tài

Đề tài hướng đến các mục tiêu chính sau:

1. Trình bày cơ sở lý thuyết của thuật toán SVM, đặc biệt là ý tưởng Maximum Margin, Soft Margin, Support Vector, bài toán đối ngẫu và Kernel Trick.
2. Xây dựng bốn mô hình SVM với Kernel Linear, RBF, Polynomial và Sigmoid.
3. Chia dữ liệu thành tập huấn luyện và kiểm thử theo tỷ lệ 80/20, đồng thời sử dụng các chỉ số Accuracy, Precision, Recall và F1-score để đánh giá.
4. Lưu mô hình và các kết quả đánh giá thành các tệp có thể được sử dụng bởi lớp API.
5. Xây dựng hệ thống web có khả năng nhận thông số hoa và trả về loài được dự đoán.
6. Bổ sung các chức năng trực quan hóa dữ liệu, Decision Boundary, SVM Playground, thí nghiệm, benchmark, phân tích tệp và lịch sử nhận diện.
7. Triển khai hệ thống trực tuyến theo mô hình frontend – backend, có endpoint Health Check để kiểm tra trạng thái dịch vụ.

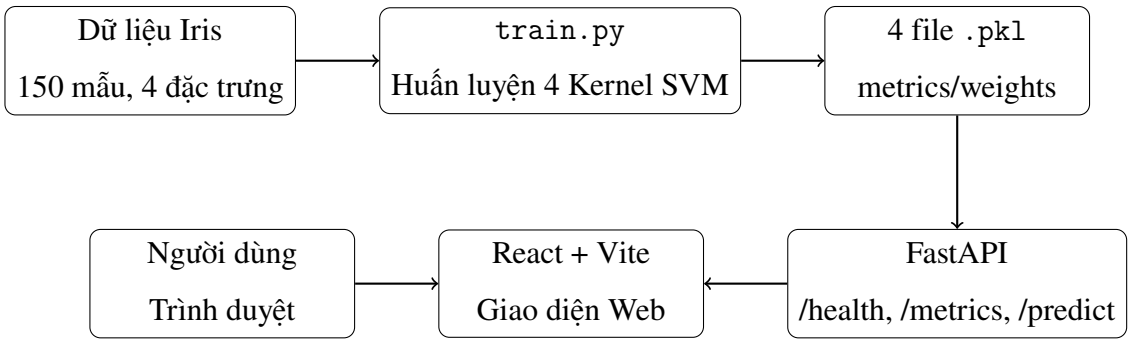
1.3. Phạm vi của báo cáo

Báo cáo tập trung vào bộ dữ liệu Fisher Iris, mô hình SVM đa lớp sử dụng thư viện scikit-learn ở lớp huấn luyện, API FastAPI ở lớp phục vụ mô hình và frontend React/Vite ở lớp giao diện. Phần triển khai trực tuyến tập trung vào cấu trúc hiện có trong repository, trong đó frontend được đóng gói bằng Vite và server Node/Express, còn API mô hình có thể được phục vụ độc lập bằng FastAPI.

1.4. Kiến trúc tổng quát của hệ thống

Hệ thống có thể được mô tả theo bốn lớp chính:

1. **Lớp dữ liệu và mô hình:** bộ dữ liệu Iris, mã huấn luyện `train.py`, bốn file mô hình `svm_linear.pkl`, `svm_rbf.pkl`, `svm_poly.pkl`, `svm_sigmoid.pkl`.
2. **Lớp API:** FastAPI tải các mô hình vào RAM, cung cấp các endpoint `/`, `/health`, `/metrics` và `/predict`.
3. **Lớp frontend:** React quản lý giao diện và các phân hệ; Vite đảm nhiệm quá trình build.
4. **Lớp lưu trữ và trạng thái:** LocalStorage/SessionStorage được dùng cho một số dữ liệu phía trình duyệt; dự án đồng thời có cấu hình Supabase cho tài khoản, lịch sử dự đoán và lịch sử thí nghiệm.



Hình 1: Kiến trúc khái quát của hệ thống Iris SVM

CHƯƠNG 2: MÔ HÌNH SVM - CƠ SỞ LÝ THUYẾT

2.1. Khái niệm Support Vector Machine

Support Vector Machine là một thuật toán học có giám sát được sử dụng cho phân loại và hồi quy. Với bài toán phân loại nhị phân, SVM tìm một siêu phẳng phân chia hai lớp sao cho khoảng cách từ siêu phẳng đến các mẫu gần nhất của hai lớp là lớn nhất. Các mẫu nằm gần ranh giới và có vai trò quyết định vị trí của siêu phẳng được gọi là Support Vectors.

Giả sử dữ liệu huấn luyện gồm các cặp $(\mathbf{x}_i, y_i)$ với $\mathbf{x}_i \in \mathbb{R}^d$ và $y_i \in \{-1, +1\}$. Một siêu phẳng có dạng:

$$\mathbf{w}^T\mathbf{x} + b = 0, \tag{1}$$

trong đó $\mathbf{w}$ là vector pháp tuyến và b là hệ số dịch chuyển.

Quy tắc phân lớp được viết:

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b). \tag{2}$$

2.2. Ý tưởng Maximum Margin

Nếu dữ liệu phân tách tuyến tính, có thể tồn tại nhiều siêu phẳng thỏa mãn điều kiện phân loại. SVM không chọn tùy ý một siêu phẳng mà tìm siêu phẳng có biên lớn nhất. Với cách chuẩn hóa $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$, khoảng cách từ một điểm đến siêu phẳng là:

$$\frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}. \tag{3}$$

Hai siêu phẳng biên là:

$$\mathbf{w}^T\mathbf{x} + b = 1, \qquad \mathbf{w}^T\mathbf{x} + b = -1. \tag{4}$$

Khoảng cách giữa hai biên bằng:

$$\frac{2}{\|\mathbf{w}\|}. \tag{5}$$

Do đó, tối đa hóa Margin tương đương với tối thiểu hóa $\|\mathbf{w}\|$. Bài toán tối ưu nguyên thủy có thể viết:

$$\min_{\mathbf{w},b} \quad \frac{1}{2}\|\mathbf{w}\|^2 \tag{6}$$

$$\text{với} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, n. \tag{7}$$

2.3. Soft Margin và tham số C

Trong dữ liệu thực tế, các lớp có thể chồng lấn hoặc có nhiễu. Khi đó, yêu cầu tất cả mẫu đều nằm đúng phía của biên là quá chặt. SVM đưa vào biến slack $\xi_i \geq 0$ để cho phép một số mẫu vi phạm biên:

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \tag{8}$$

$$\text{với} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \tag{9}$$

Tham số C điều chỉnh sự đánh đổi giữa độ rộng Margin và mức phạt lỗi. Khi C lớn, mô hình có xu hướng phạt mạnh các điểm vi phạm và cố gắng phân loại đúng nhiều mẫu huấn luyện hơn. Khi C nhỏ, mô hình chấp nhận nhiều vi phạm hơn để có một Margin rộng hơn.

2.4. Hàm mất mát Hinge

Soft Margin SVM có thể liên hệ với Hinge Loss:

$$L(y, f(\mathbf{x})) = \max(0, 1 - yf(\mathbf{x})). \tag{10}$$

Nếu một điểm được phân loại đúng và nằm ngoài biên, Hinge Loss bằng 0. Nếu điểm nằm trong biên hoặc bị phân loại sai, mô hình phải chịu một mức phạt tương ứng.

2.5. Dạng đối ngẫu và Support Vector

Thông qua phương pháp nhân tử Lagrange, bài toán có thể chuyển sang dạng đối ngẫu:

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j \tag{11}$$

$$\text{s.t.} \quad \alpha_i \geq 0, \qquad \sum_{i=1}^{n} \alpha_i y_i = 0. \tag{12}$$

Các mẫu có $\alpha_i > 0$ là những mẫu quan trọng đối với nghiệm tối ưu và được gọi là Support Vectors. Vì vậy, SVM không cần mọi điểm dữ liệu đóng góp như nhau vào phương trình quyết định.

Hàm quyết định có dạng:

$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b. \tag{13}$$

Đây chính là cơ sở để đưa Kernel vào SVM.

2.6. Kernel Trick

Trong trường hợp dữ liệu không thể phân tách tuyến tính trong không gian ban đầu, ta có thể ánh xạ dữ liệu sang một không gian đặc trưng mới thông qua ánh xạ $\phi(\mathbf{x})$. Thay vì tính trực tiếp $\phi(\mathbf{x})$, Kernel Trick sử dụng:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j). \tag{14}$$

Nhờ đó, mô hình có thể tạo ranh giới phi tuyến mà không nhất thiết phải xây dựng tường minh vector đặc trưng trong không gian có chiều rất cao.

2.7. Bốn Kernel được sử dụng trong đề tài

2.7.1. Kernel Linear

$$K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}. \tag{15}$$

Kernel Linear phù hợp khi các lớp có thể phân chia tương đối tốt bằng siêu phẳng. Trong mã huấn luyện, mô hình được tạo với $C = 1.0$.

2.7.2. Kernel RBF

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right). \tag{16}$$

RBF là Kernel phi tuyến phổ biến. Trong dự án, $C = 1.0$ và $\gamma =$ `scale`. Khi sử dụng `scale`, scikit-learn xác định gamma dựa trên phương sai của dữ liệu và số chiều đặc trưng.

2.7.3. Kernel Polynomial

$$K(\mathbf{x}, \mathbf{z}) = (\gamma \mathbf{x}^T \mathbf{z} + r)^d. \tag{17}$$

Trong dự án, Polynomial được cấu hình với $C = 1.0$, bậc $d = 3$, $\gamma =$ `scale` và $r = 1.0$ thông qua tham số `coef0`.

2.7.4. Kernel Sigmoid

$$K(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \mathbf{x}^T \mathbf{z} + r). \tag{18}$$

Trong cấu hình huấn luyện, Sigmoid sử dụng $C = 1.0$, $\gamma =$ `scale` và $r = 0.0$.

2.8. SVM đa lớp

Bộ dữ liệu Iris có ba lớp. Trong scikit-learn, `SVC` hỗ trợ bài toán đa lớp bằng cách xây dựng các bộ phân loại nhị phân theo chiến lược One-vs-One. Với $K = 3$ lớp, số bộ phân loại cặp lớp là:

$$\frac{K(K-1)}{2} = \frac{3 \times 2}{2} = 3. \tag{19}$$

Kết quả cuối cùng được tổng hợp từ các bộ phân loại nhị phân để xác định lớp dự đoán.

2.9. Các chỉ số đánh giá

Accuracy đo tỷ lệ dự đoán đúng trên tổng số mẫu:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \tag{20}$$

Trong bài toán đa lớp, báo cáo sử dụng Precision, Recall và F1 theo trung bình Macro. Với từng lớp:

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}. \tag{21}$$

F1-score được tính:

$$F1 = 2\frac{Precision \times Recall}{Precision + Recall}. \tag{22}$$

Macro-average lấy trung bình chỉ số của ba lớp, do đó mỗi loài có trọng số như nhau.

CHƯƠNG 3: DỮ LIỆU VÀ TRỰC QUAN HÓA DỮ LIỆU

3.1. Giới thiệu bộ dữ liệu Iris

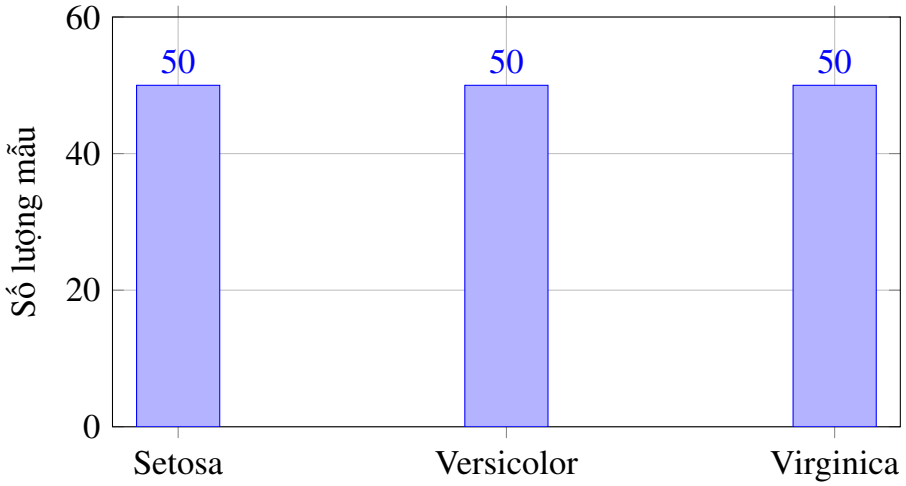
Bộ dữ liệu Fisher Iris gồm 150 mẫu thuộc ba loài: *Iris setosa*, *Iris versicolor* và *Iris virginica*. Mỗi lớp có 50 mẫu. Mỗi mẫu có bốn đặc trưng liên tục, đơn vị centimet.

Bảng 1: Các đặc trưng của bộ dữ liệu Iris

STT	Đặc trưng	Ý nghĩa
1	Sepal Length	Chiều dài đài hoa
2	Sepal Width	Chiều rộng đài hoa
3	Petal Length	Chiều dài cánh hoa
4	Petal Width	Chiều rộng cánh hoa

3.2. Phân bố số lượng mẫu

Dữ liệu có cấu trúc cân bằng hoàn toàn: mỗi lớp gồm 50 mẫu. Vì vậy, việc sử dụng Macro-average cho Precision, Recall và F1 là phù hợp để quan sát chất lượng trên từng loài mà không để lớp nào có trọng số lớn hơn chỉ vì có nhiều mẫu hơn.



Hình 2: Phân bố mẫu theo ba lớp của bộ dữ liệu Iris

3.3. Thống kê mô tả

Các thống kê dưới đây được tính trực tiếp từ 150 mẫu được lưu trong `src/data/irisData.ts`. Độ lệch chuẩn được tính theo công thức của tổng thể với mẫu số bằng số lượng phần tử, phù hợp với cách tính thống kê trong hàm `getFeatureStats` của frontend.

Bảng 2: Thống kê theo từng loài Iris

Loài	Đặc trưng	Mean	Std	Min	Max
Setosa	Sepal Length	5.006	0.349	4.3	5.8
	Sepal Width	3.428	0.375	2.3	4.4
	Petal Length	1.462	0.172	1.0	1.9
	Petal Width	0.246	0.104	0.1	0.6
Versicolor	Sepal Length	5.936	0.511	4.9	7.0
	Sepal Width	2.770	0.311	2.0	3.4
	Petal Length	4.260	0.465	3.0	5.1
	Petal Width	1.326	0.196	1.0	1.8
Virginica	Sepal Length	6.588	0.629	4.9	7.9
	Sepal Width	2.974	0.319	2.2	3.8
	Petal Length	5.552	0.546	4.5	6.9
	Petal Width	2.026	0.272	1.4	2.5

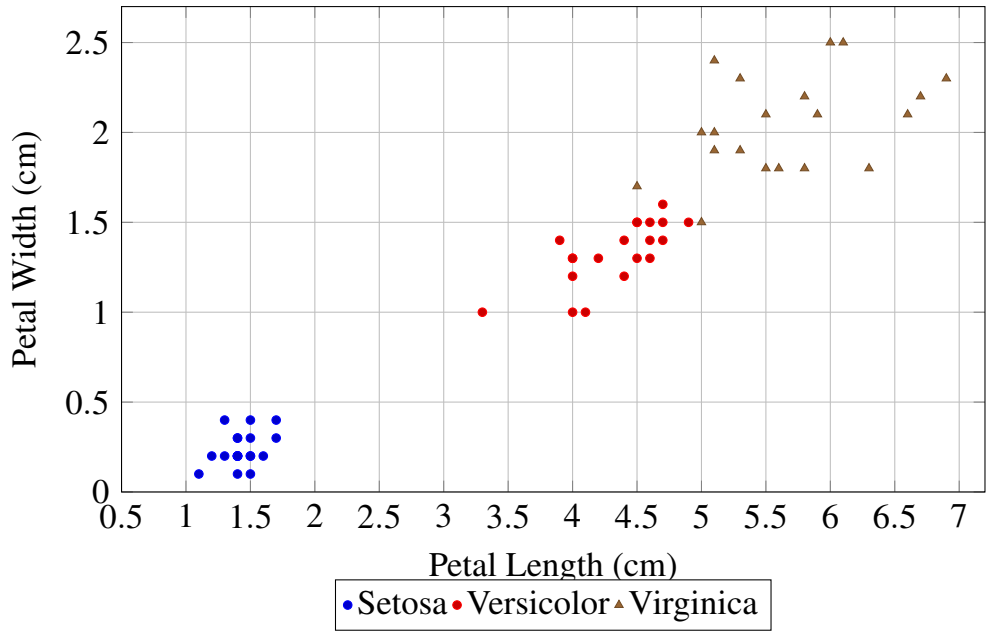
3.4. Nhận xét về khả năng phân tách

Bảng 2 cho thấy Petal Length và Petal Width có sự khác biệt rõ rệt giữa Setosa với hai lớp còn lại. Setosa có Petal Length trung bình 1.462 cm, trong khi Versicolor và Virginica lần lượt có giá trị trung bình 4.260 cm và 5.552 cm. Petal Width cũng có xu hướng tương tự.

Versicolor và Virginica gần nhau hơn, đặc biệt xét theo một số vùng giá trị của Petal Length và Petal Width. Đây là lý do trực quan hóa hai đặc trưng này thường cho thấy Setosa tách biệt khá rõ, còn ranh giới giữa Versicolor và Virginica khó hơn.

3.5. Trực quan hóa hai chiều

Frontend của dự án có phân hệ "Trực quan hóa Iris", cho phép khám phá phân bố 2D theo các cặp đặc trưng. Ngoài ra, trang Nhận diện hiển thị biểu đồ Petal Length so với Petal Width và đánh dấu vị trí mẫu hiện tại. SVM Playground cho phép lựa chọn hai đặc trưng và hiển thị Decision Boundary tương ứng.



Hình 3: Minh họa không gian Petal Length–Petal Width

Hình 3 chỉ nhằm minh họa xu hướng phân bố; giao diện web sử dụng toàn bộ 150 mẫu để vẽ biểu đồ tương tác. Trong báo cáo, việc đưa một phần điểm vào hình giúp tài liệu gọn và dễ đọc hơn, trong khi các thống kê và kết quả đánh giá vẫn sử dụng toàn bộ dữ liệu.

CHƯƠNG 4: HUẤN LUYỆN MÔ HÌNH SVM: QUY TRÌNH VÀ KIỂM TRA TÍNH HIỆU QUẢ

4.1. Quy trình huấn luyện

Quy trình huấn luyện được triển khai trong `train.py` theo các bước:

1. Tải bộ dữ liệu Iris từ `sklearn.datasets.load_iris()`.
2. Tách ma trận đặc trưng X và nhãn y.
3. Chia dữ liệu thành tập Train và Test theo tỷ lệ 80/20.
4. Sử dụng `random_state=42` để kết quả chia dữ liệu có thể tái lập.
5. Sử dụng `stratify=y` để duy trì tỷ lệ ba lớp trong cả hai tập.
6. Lặp qua bốn Kernel Linear, RBF, Polynomial và Sigmoid.
7. Huấn luyện bằng `SVC` của scikit-learn.
8. Dự đoán trên tập kiểm thử.
9. Tính Accuracy, Macro Precision, Macro Recall, Macro F1 và Confusion Matrix.
10. Đếm Support Vectors và đo thời gian huấn luyện, thời gian dự đoán.
11. Lưu mô hình bằng joblib và lưu kết quả vào `metrics.json`, `weights.json`.

4.2. Cấu hình bốn mô hình

Bảng 3: Cấu hình các mô hình SVM trong dự án

Kernel	C	γ	d	`coef0`	**File mô hình**
Linear	1.0	–	–	–	`svm_linear.pkl`
RBF	1.0	scale	–	–	`svm_rbf.pkl`
Polynomial	1.0	scale	3	1.0	`svm_poly.pkl`
Sigmoid	1.0	scale	–	0.0	`svm_sigmoid.pkl`

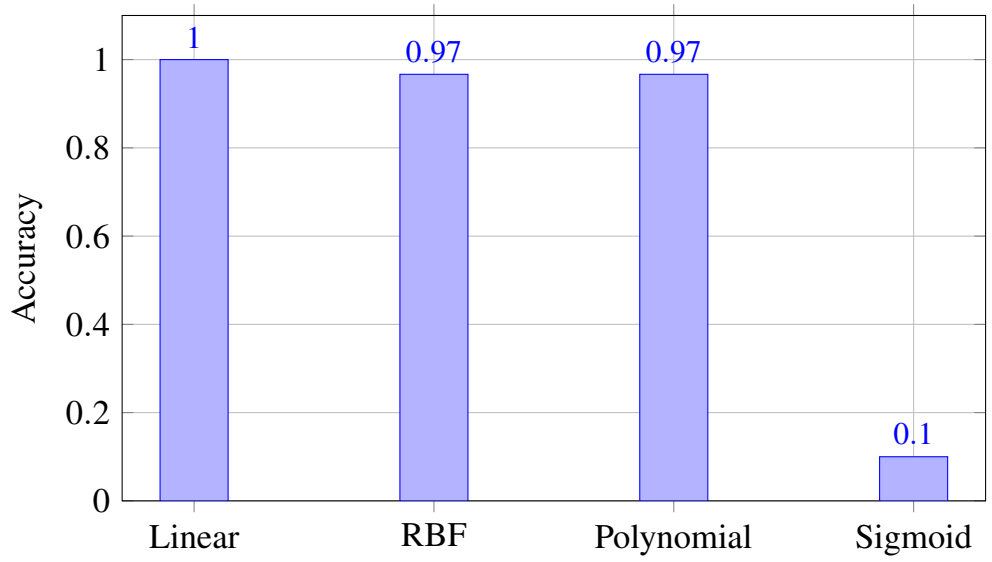
4.3. Kết quả đánh giá thực tế

Các kết quả dưới đây lấy trực tiếp từ `metrics.json` được lưu trong repository sau lần huấn luyện. Tập kiểm thử gồm 30 mẫu, tức 10 mẫu cho mỗi lớp.

Bảng 4: Kết quả đánh giá bốn Kernel SVM trên tập kiểm thử

Kernel	**Acc.**	**Prec.**	**Recall**	**F1**	**SV**	**Train (ms)**	**Pred (ms)**
Linear	1.0000	1.0000	1.0000	1.0000	23	1.36	0.22
RBF	0.9667	0.9697	0.9667	0.9666	49	1.13	0.22
Polynomial	0.9667	0.9697	0.9667	0.9666	16	0.89	0.14
Sigmoid	0.1000	0.0500	0.1000	0.0667	120	1.68	0.30

4.4. Biểu đồ so sánh Accuracy



Hình 4: So sánh Accuracy của bốn mô hình SVM

Kết quả cho thấy ba mô hình Linear, RBF và Polynomial hoạt động tốt trên phép chia dữ liệu hiện tại, trong khi Sigmoid cho kết quả thấp. Điều này không có nghĩa một Kernel luôn tốt hoặc xấu trong mọi bài toán; kết quả phụ thuộc vào dữ liệu, siêu tham số và cách tiền xử lý. Trong phạm vi thí nghiệm này, cấu hình Sigmoid với $C = 1.0$, $\gamma =$ `scale` và `coef0`=0 không phù hợp với phép chia dữ liệu đang sử dụng.

4.5. Confusion Matrix

Bảng 5: Confusion Matrix của Kernel Linear

Thực tế / Dự đoán	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	10	0
Virginica	0	0	10

Bảng 6: Confusion Matrix của Kernel RBF và Polynomial

Thực tế / Dự đoán	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	0	10

RBF và Polynomial đều có một mẫu Versicolor bị dự đoán thành Virginica. Không có mẫu Setosa bị nhầm trong phép thử này. Với 30 mẫu kiểm thử, một lỗi tương ứng Accuracy $= 29/30 \approx 0.9667$.

Bảng 7: Confusion Matrix của Kernel Sigmoid

Thực tế / Dự đoán	Setosa	Versicolor	Virginica
Setosa	3	0	7
Versicolor	7	0	3
Virginica	10	0	0

Ma trận của Sigmoid cho thấy mô hình không dự đoán được lớp Versicolor trong tập kiểm thử và có xu hướng đẩy nhiều mẫu về Setosa hoặc Virginica. Đây là một dấu hiệu rõ ràng cho thấy cấu hình hiện tại chưa phù hợp với dữ liệu.

4.6. Support Vector và thời gian thực thi

Số Support Vector của bốn mô hình lần lượt là 23, 49, 16 và 120. Sigmoid sử dụng 120 Support Vector, tức toàn bộ số mẫu của tập huấn luyện, cho thấy nghiệm của mô hình ở cấu hình này phụ thuộc vào rất nhiều điểm dữ liệu.

Thời gian huấn luyện đều ở mức mili-giây đối với bộ dữ liệu nhỏ. Polynomial có thời gian huấn luyện 0.89 ms và thời gian dự đoán 0.14 ms trong lần chạy được lưu. Các số đo này có thể thay đổi theo phần cứng, môi trường Python và tải hệ thống, do đó không nên xem là hằng số tuyệt đối.

4.7. Kiểm tra tính hiệu quả

Việc kiểm tra tính hiệu quả được thực hiện theo nhiều góc độ:

- **Hiệu quả phân loại:** sử dụng Accuracy, Precision, Recall và F1-score.
- **Khả năng nhận diện từng lớp:** sử dụng Confusion Matrix.
- **Độ phức tạp nghiệm:** quan sát số lượng Support Vector.
- **Hiệu quả tính toán:** đo thời gian huấn luyện và dự đoán.
- **Khả năng vận hành:** sau khi huấn luyện, mô hình được lưu thành file và nạp vào API để phục vụ nhiều yêu cầu dự đoán.

4.8. Nhận xét kết quả thực nghiệm

Từ kết quả thực nghiệm trên tập kiểm thử gồm 30 mẫu, có thể thấy hiệu quả của bốn Kernel SVM có sự khác biệt rõ rệt. Các mô hình được huấn luyện trên cùng một tập dữ liệu, cùng cách chia dữ liệu theo tỷ lệ 80/20 và cùng giá trị $C = 1.0$, qua đó tạo điều kiện thuận lợi cho việc so sánh.

Kernel Linear đạt Accuracy, Precision, Recall và F1 đều bằng 1.0000. Ma trận nhầm lẫn cho thấy cả 30 mẫu trong tập kiểm thử đều được phân loại chính xác. Mô hình sử dụng 23 Support Vector, thời gian huấn luyện là 1.36 ms và thời gian dự đoán là 0.22 ms.

Kernel RBF đạt Accuracy 0.9667, Precision 0.9697, Recall 0.9667 và F1 0.9666. Ma trận nhầm lẫn cho thấy có một mẫu thuộc lớp Versicolor bị dự đoán thành lớp Virginica. Mô hình sử dụng 49 Support Vector, thời gian huấn luyện là 1.13 ms và thời gian dự đoán là 0.22 ms.

Kernel Polynomial cũng đạt Accuracy 0.9667, Precision 0.9697, Recall 0.9667 và F1 0.9666. Tương tự Kernel RBF, mô hình có một mẫu thuộc lớp Versicolor bị phân loại nhầm sang lớp Virginica. Tuy nhiên, số lượng Support Vector của Kernel Polynomial chỉ là 16, thấp hơn so với RBF. Thời gian huấn luyện là 0.89 ms và thời gian dự đoán là 0.14 ms.

Kernel Sigmoid cho kết quả thấp hơn đáng kể với Accuracy 0.1000, Precision 0.0500, Recall 0.1000 và F1 0.0667. Mô hình sử dụng 120 Support Vector, tương ứng với toàn bộ số mẫu của tập huấn luyện. Thời gian huấn luyện là 1.68 ms và thời gian dự đoán là 0.30 ms. Kết quả này cho thấy cấu hình Kernel Sigmoid được sử dụng trong thí nghiệm chưa phù hợp với dữ liệu Iris và các siêu tham số hiện tại.

Nhìn chung, thí nghiệm cho thấy hiệu quả của SVM phụ thuộc đáng kể vào việc lựa chọn Kernel và các siêu tham số đi kèm. Với cùng một tập dữ liệu và cùng phương pháp chia Train/Test, bốn Kernel cho các kết quả khác nhau về độ chính xác, số lượng Support Vector cũng như thời gian tính toán. Do đó, việc thử nghiệm nhiều Kernel là cần thiết để đánh giá mức độ phù hợp của mô hình đối với một bài toán phân loại cụ thể.

Trong phạm vi của đề tài, kết quả thực nghiệm được sử dụng để minh họa và so sánh trực tiếp bốn Kernel Linear, RBF, Polynomial và Sigmoid trên bộ dữ liệu Iris. Các kết quả được trình bày trong báo cáo được lấy từ tệp `metrics.json` được sinh ra sau quá trình huấn luyện mô hình.

CHƯƠNG 5: CÁC CÔNG NGHỆ SỬ DỤNG ĐỂ TRIỂN KHAI TRỰC TUYẾN

5.1. Python và scikit-learn

Python được sử dụng cho phần huấn luyện mô hình. Thư viện scikit-learn cung cấp `SVC`, `train_test_split` và các hàm đánh giá. Đây là lớp tính toán chính tạo ra bốn mô hình SVM.

5.2. Joblib

Sau khi huấn luyện, mô hình được lưu bằng `joblib.dump`. Mỗi Kernel có một file riêng. Khi API khởi động, các file này được tải vào RAM bằng `joblib.load`. Cách làm này tránh phải huấn luyện lại mỗi khi có một yêu cầu dự đoán.

5.3. FastAPI

FastAPI được sử dụng để xây dựng API phục vụ mô hình. Ứng dụng khai báo tên "Iris Multi-Kernel SVM API", phiên bản 2.0.0 và hỗ trợ bốn Kernel.

API chính gồm:

- `GET /`: kiểm tra API đang hoạt động và trả về danh sách Kernel.
- `GET /health`: kiểm tra tình trạng dịch vụ và số mô hình đã được tải.
- `GET /metrics`: trả về các kết quả đánh giá đã lưu.
- `POST /predict`: nhận bốn đặc trưng và Kernel, sau đó trả về lớp dự đoán và thời gian thực thi.

5.4. Pydantic

Dữ liệu đầu vào API được mô tả bằng lớp `IrisInput`. Bốn đặc trưng đều có kiểu số thực và Kernel được giới hạn bởi `Literal` vào bốn giá trị `linear`, `rbf`, `poly`, `sigmoid`. Điều này giúp cấu trúc dữ liệu đầu vào rõ ràng và giảm nguy cơ truyền giá trị Kernel không hợp lệ.

5.5. React và Vite

Frontend được tổ chức bằng React với nhiều component. Vite được dùng để build ứng dụng. Các phân hệ chính được khai báo trong `App.tsx` và được chuyển đổi bằng trạng thái `activeTab`, vì vậy hệ thống hoạt động theo mô hình Single Page Application.

5.6. Node.js và Express

Repository có `server.ts` sử dụng Express để phục vụ ứng dụng và tích hợp middleware của Vite trong môi trường phát triển. Khi chạy production, server phục vụ thư mục `dist`. Server cũng có endpoint `/health` và `/metrics` cho lớp frontend.

5.7. Supabase

Dự án có tích hợp Supabase ở lớp dữ liệu ứng dụng. Schema hiện có các bảng như `app_users`, `profiles`, `user_preferences`, `prediction_history`, `experiment_history` và `app_content`. Mục tiêu là lưu tài khoản, cấu hình, lịch sử dự đoán và lịch sử thí nghiệm.

5.8. Render

Cấu hình `render.yaml` của frontend khai báo một Web Service chạy Node, sử dụng:

- Build Command: `npm install && npm run build`.
- Start Command: `npm start`.
- Health Check Path: `/health`.
- Auto Deploy Trigger: commit.

5.9. Kiến trúc triển khai thực tế

Frontend trong `src/utils/api.ts` gọi API FastAPI tại endpoint Render được cấu hình sẵn. Khi API từ xa phản hồi thành công, kết quả được sử dụng. Nếu API không phản hồi trong thời gian chờ khoảng 4.5 giây hoặc xảy ra lỗi mạng, frontend chuyển sang hàm dự đoán cục bộ. Cơ chế này giúp giao diện vẫn có thể hoạt động trong tình huống API từ xa đang cold start hoặc tạm thời không truy cập được.

CHƯƠNG 6: CHI TIẾT QUY TRÌNH TRIỂN KHAI TRỰC TUYẾN VÀ VẬN HÀNH

Giai đoạn 1 – Huấn luyện và tạo artifact

Trước khi triển khai, chạy chương trình huấn luyện để tạo bốn file mô hình:

1. `svm_linear.pkl`
2. `svm_rbf.pkl`
3. `svm_poly.pkl`
4. `svm_sigmoid.pkl`

Đồng thời tạo:

- `metrics.json`: kết quả Accuracy, Precision, Recall, F1, Support Vector, thời gian huấn luyện và ma trận nhầm lẫn.
- `weights.json`: thông tin mở rộng gồm cấu hình tham số, dữ liệu, thời gian huấn luyện và metrics.

Giai đoạn 2 – Kiểm tra API cục bộ

Sau khi tạo artifact, API cần được chạy cục bộ và kiểm tra các endpoint. Ví dụ một yêu cầu dự đoán có dạng:

Listing 1: Dữ liệu JSON gửi đến endpoint /predict

```
{
        "sepal_length": 5.1,
        "sepal_width": 3.5,
        "petal_length": 1.4,
        "petal_width": 0.2,
        "kernel": "linear"
}
```

API trả về thông tin Kernel được sử dụng, mã lớp, tên loài, thời gian thực thi và các đặc trưng đầu vào. Với mẫu trên, lớp dự đoán tương ứng là *setosa* theo mô hình Linear trong bộ mô hình đã huấn luyện.

Giai đoạn 3 – Kiểm tra Health Check

Endpoint `/health` có vai trò quan trọng khi triển khai. Một phản hồi hợp lệ chứa trạng thái `healthy`, danh sách Kernel đã được nạp và tổng số mô hình. Điều này cho phép nền tảng triển khai hoặc người quản trị phát hiện trường hợp server hoạt động nhưng mô hình chưa được tải đầy đủ.

Giai đoạn 4 – Đóng gói frontend

Frontend sử dụng Node.js với các script:

- `npm run dev`: chạy môi trường phát triển.
- `npm run build`: build ứng dụng Vite.
- `npm start`: chạy server Express/tsx.
- `npm run lint`: kiểm tra TypeScript.

Giai đoạn 5 – Đẩy repository và triển khai Render

Quy trình vận hành triển khai frontend có thể thực hiện theo trình tự:

1. Kiểm tra code và artifact cần thiết trong repository.
2. Commit thay đổi.
3. Push lên nhánh repository được Render theo dõi.
4. Render thực hiện build theo `npm install && npm run build`.
5. Render khởi động bằng `npm start`.
6. Render kiểm tra endpoint `/health`.
7. Sau khi service sẵn sàng, người dùng truy cập địa chỉ web được Render cung cấp.

Giai đoạn 6 – Vận hành sau triển khai

Trong quá trình vận hành, cần kiểm tra bốn nhóm vấn đề:

1. **Khả dụng:** trang web có tải được hay không.
2. **API:** endpoint dự đoán có trả về JSON đúng cấu trúc hay không.
3. **Mô hình:** bốn file model có được tải và Kernel có đúng với yêu cầu không.
4. **Dữ liệu:** metrics và lịch sử có được ghi nhận theo thiết kế hay không.

Luồng một yêu cầu dự đoán

Luồng xử lý có thể mô tả như sau:

Người dùng nhập dữ liệu → React tạo JSON → POST /predict
→ FastAPI chọn Kernel → SVM.predict() → JSON kết quả → React hiển thị kết quả

Trong trường hợp API từ xa không phản hồi trong khoảng thời gian timeout, frontend gọi mô hình SVM cục bộ thông qua `predictIrisLocally`. Đây là cơ chế dự phòng nhằm tăng tính liên tục của trải nghiệm người dùng.

Lưu ý về bảo mật và vận hành

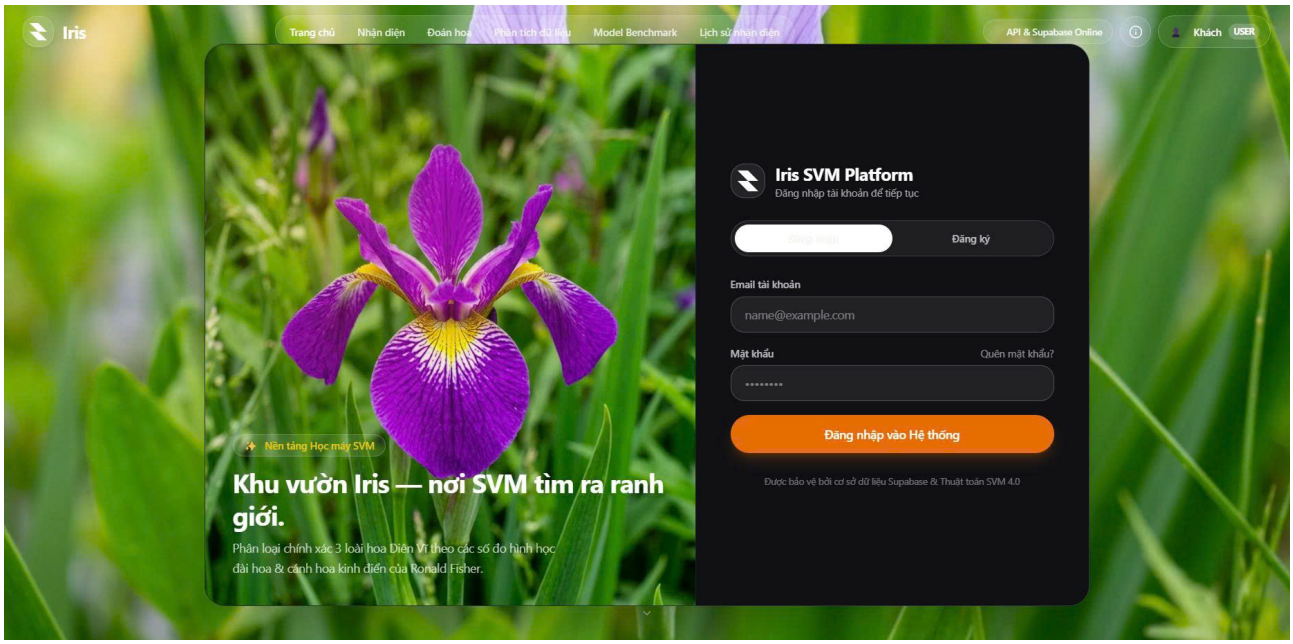
Schema Supabase hiện tại trong repository có thiết kế đơn giản hóa đăng nhập, lưu trường mật khẩu trực tiếp và tắt Row Level Security. Đây là cấu hình phù hợp cho mục đích học tập hoặc thử nghiệm nội bộ nhưng không nên được xem là cấu hình bảo mật cho hệ thống production. Khi triển khai thực tế cần sử dụng hệ thống xác thực chuẩn, không lưu mật khẩu dạng plaintext, bật RLS và giới hạn quyền truy cập theo người dùng.

Ngoài ra, các khóa môi trường như `VITE_SUPABASE_URL` và `VITE_SUPABASE_PUBLISHABLE_KEY` phải được cấu hình qua biến môi trường; không nên đưa secret server-side vào mã nguồn frontend.

CHƯƠNG 7: DEMO VÀ HƯỚNG DẪN SỬ DỤNG WEB CHO NGƯỜI DÙNG

7.1. Tổng quan quy trình truy cập và Giao diện chính

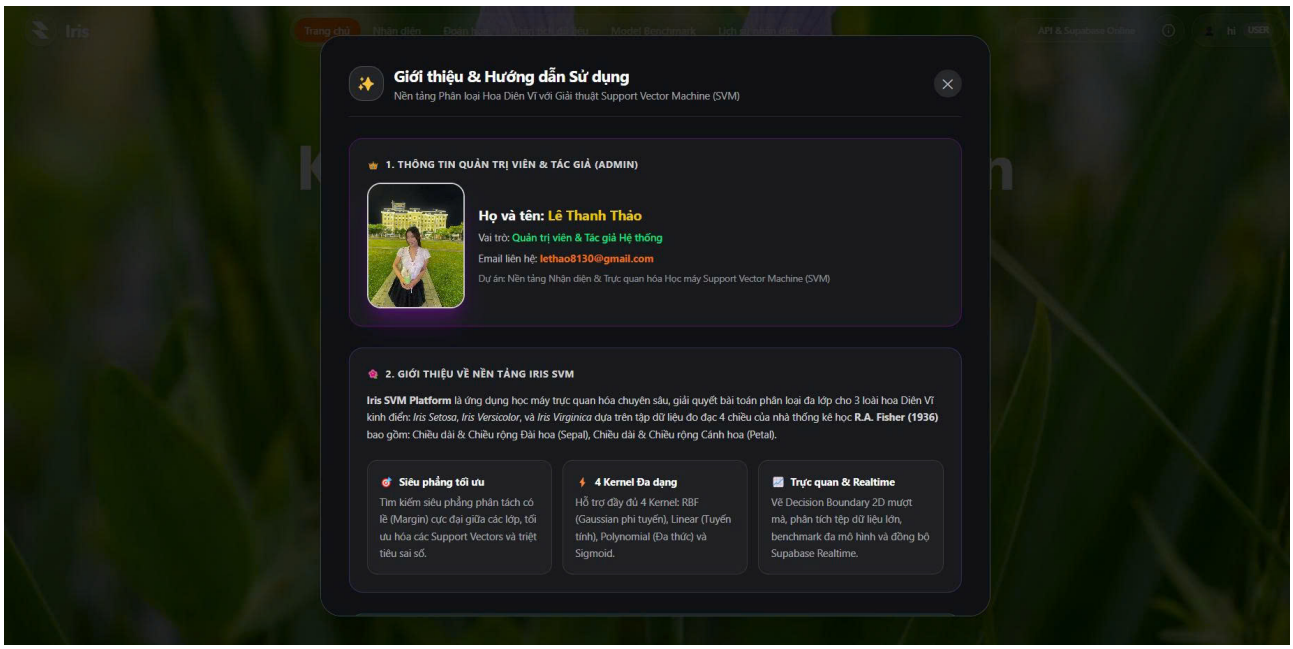
Hệ thống được thiết kế với giao diện hiện đại, tập trung vào trải nghiệm người dùng, giúp tối ưu hóa quy trình từ khâu xác thực đến khâu sử dụng các tính năng phân tích chi tiết.



Hình 5: Giao diện cổng đăng nhập của Nền tảng Iris SVM

Hình 5 minh họa **Cổng đăng nhập**, điểm khởi đầu của hành trình khám phá "Khu vườn Iris". Tại đây, sức mạnh của thuật toán Học máy SVM được tích hợp ngầm để phân loại chính xác 3 loài hoa Diên vĩ dựa trên bộ dữ liệu kinh điển của Ronald Fisher. Để đảm bảo an toàn dữ liệu và cá nhân hóa trải nghiệm, người dùng cần thực hiện Đăng ký hoặc Đăng nhập. Hệ thống sử dụng Supabase ở lớp dữ liệu ứng dụng để hỗ trợ quản lý tài khoản, cấu hình người dùng, lịch sử dự đoán và lịch sử thí nghiệm. Trong phiên bản hiện tại, cơ chế này chủ yếu phục vụ mục đích học tập và minh họa kiến trúc hệ thống; các yêu cầu bảo mật production cần được bổ sung trước khi triển khai thực tế.

Sau khi xác thực tài khoản thành công, hệ thống sẽ tự động chuyển hướng người dùng đến **Trang chào mừng và giới thiệu**.



Hình 6: Giao diện điều hướng chính và Thông tin tác giả

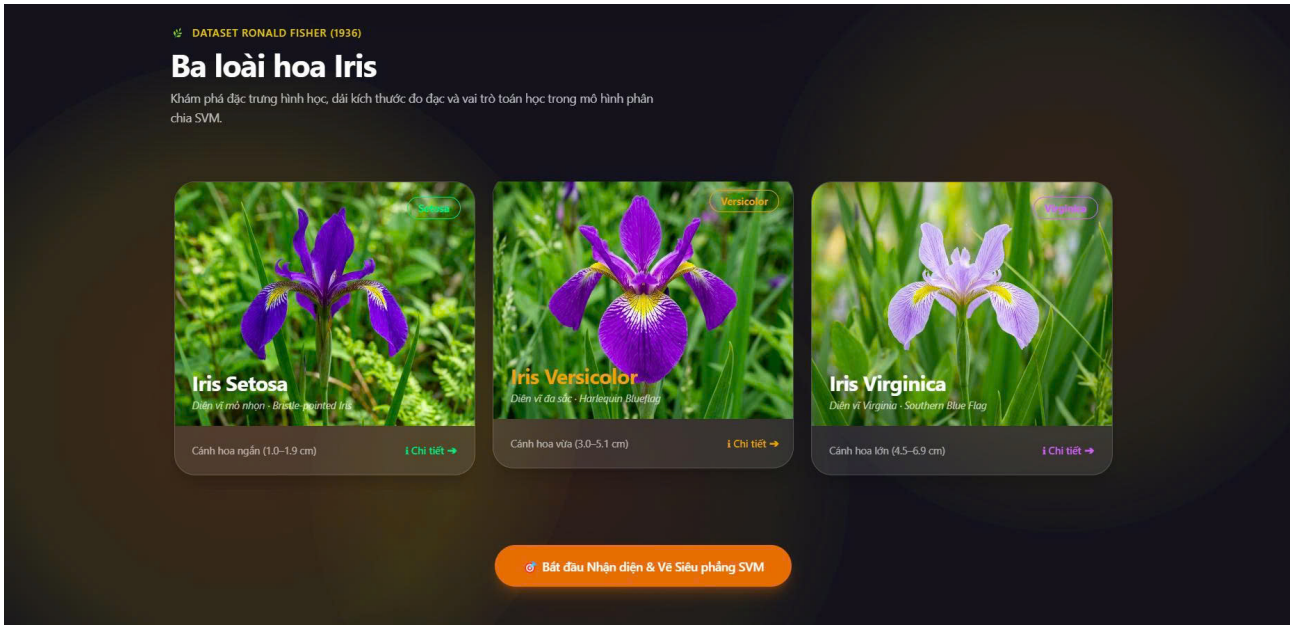
Giao diện chính là **Trang chủ**, được thiết kế tối giản nhưng ấn tượng với hình ảnh hoa Diên vĩ làm nền và slogan "Khám phá ranh giới phân loại bằng SVM". Tại đây, người dùng có thể thực hiện hai thao tác chính: sử dụng thanh điều hướng phía trên để truy cập nhanh các phân hệ chức năng, hoặc cuộn trang xuống để bắt đầu tìm hiểu về bộ dữ liệu.



Hình 7: Khám phá ranh giới phân loại bằng SVM

Khi người dùng cuộn chuột xuống phía dưới, hệ thống sẽ hiển thị phần giới thiệu chi tiết về bộ dữ liệu kinh điển của Ronald Fisher. Tại đây, ba loài hoa Iris chính (Setosa, Versicolor, và Virginica) được trực quan hóa thông qua các thẻ *card* thông tin riêng biệt. Mỗi thẻ cung cấp

hình ảnh thực tế, tên khoa học và thông tin tóm tắt về đặc trưng kích thước cánh hoa đặc trưng của từng loài.



Hình 8: Khám phá 3 loài hoa Iris

Để tìm hiểu sâu hơn về đặc điểm hình thái của một loài cụ thể, người dùng có thể nhấn vào nút **"Chi tiết"** có biểu tượng mũi tên xanh trên thẻ tương ứng. Ngay lập tức, hệ thống sẽ hiển thị một cửa sổ thông tin đầy đủ và chi tiết hơn (như hình minh họa 8), bao gồm các số liệu đo lường cụ thể về chiều dài/chiều rộng đài hoa, cánh hoa, giúp người dùng có cái nhìn tổng quan trước khi tiến hành các bước nhận diện tiếp theo.

Và sau khi bấm *Bắt đầu Nhận diện & Vẽ siêu phẳng SVM* , hệ thống hiển thị thanh điều hướng với tám phân hệ:

1. Nhận diện.
2. Đoán hoa
3. Phân tích dữ liệu.
4. Model Benchmark
5. Lịch sử nhận diện.
6. Thí nghiệm toàn hệ thống (Nếu là admin).
7. Quản trị & Thống kê (Nếu là admin).

Giao diện sử dụng cơ chế xác minh phiên bằng SessionStorage trước khi người dùng truy cập đầy đủ hệ thống. Sau khi xác minh, người dùng có thể chuyển đổi giữa các phân hệ mà không cần tải lại toàn bộ trang.

7.1.1. Chức năng 1 – Nhận diện & Huấn luyện SVM

Đây là chức năng chính dành cho người dùng phổ thông.

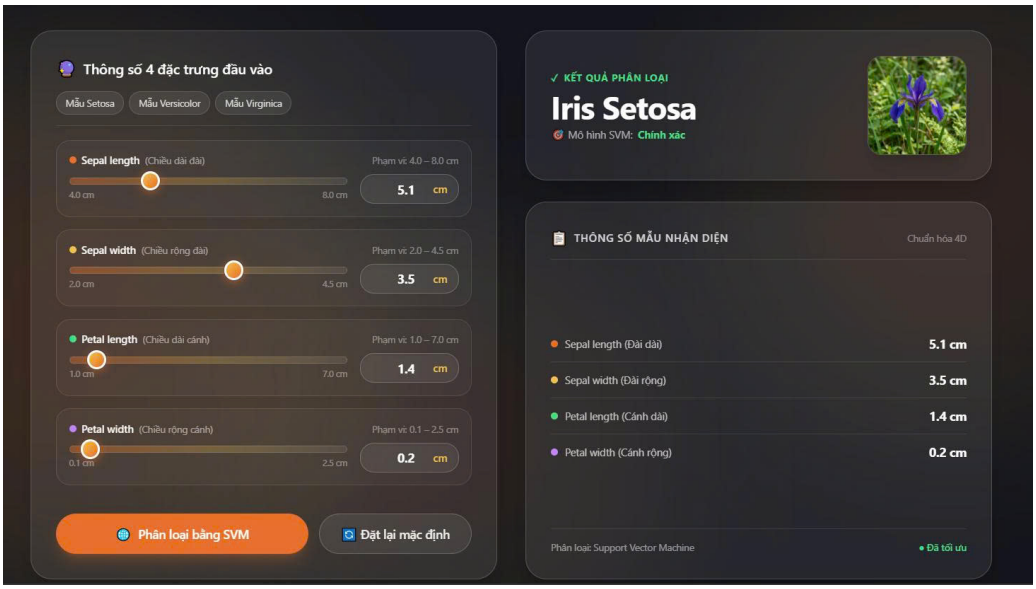
Bước 1: Nhập thông số. Người dùng nhập bốn giá trị:

- Sepal Length, khoảng giao diện từ 4 đến 8 cm.
- Sepal Width, khoảng giao diện từ 2 đến 4.5 cm.
- Petal Length, khoảng giao diện từ 1 đến 7 cm.
- Petal Width, khoảng giao diện từ 0.1 đến 2.5 cm.

Mỗi đặc trưng có cả thanh kéo và ô nhập số, giúp người dùng có thể thao tác nhanh hoặc nhập chính xác.

Bước 2: Nhấn "Phân loại bằng SVM". Frontend gửi dữ liệu đến API. Kết quả được chuẩn hóa về một trong ba nhãn: Setosa, Versicolor hoặc Virginica.

Bước 3: Xem kết quả. Hệ thống hiển thị tên loài, độ tin cậy được giao diện tính/nhận, ảnh minh họa và bốn thông số đầu vào.



Hình 9: Chức năng Nhận diện

Bước 4: Kiểm tra trực quan. Bên dưới kết quả có biểu đồ Petal Length–Petal Width và vị trí của mẫu hiện tại. Người dùng có thể đối chiếu vị trí mẫu với vùng phân bố dữ liệu.

Bước 5: Đặt lại. Nút "Đặt lại" đưa các thông số về mẫu mặc định 5.1, 3.5, 1.4, 0.2 và kết quả Setosa.

Ví dụ Demo 1 – Mẫu Setosa

Sử dụng:

Sepal Length	5.1 cm
Sepal Width	3.5 cm
Petal Length	1.4 cm
Petal Width	0.2 cm
Kernel API	Linear

Với mô hình Linear được huấn luyện trong dự án, mẫu được phân loại thành *Iris setosa*. Đây cũng là mẫu mặc định được sử dụng trong nhiều thành phần của hệ thống.

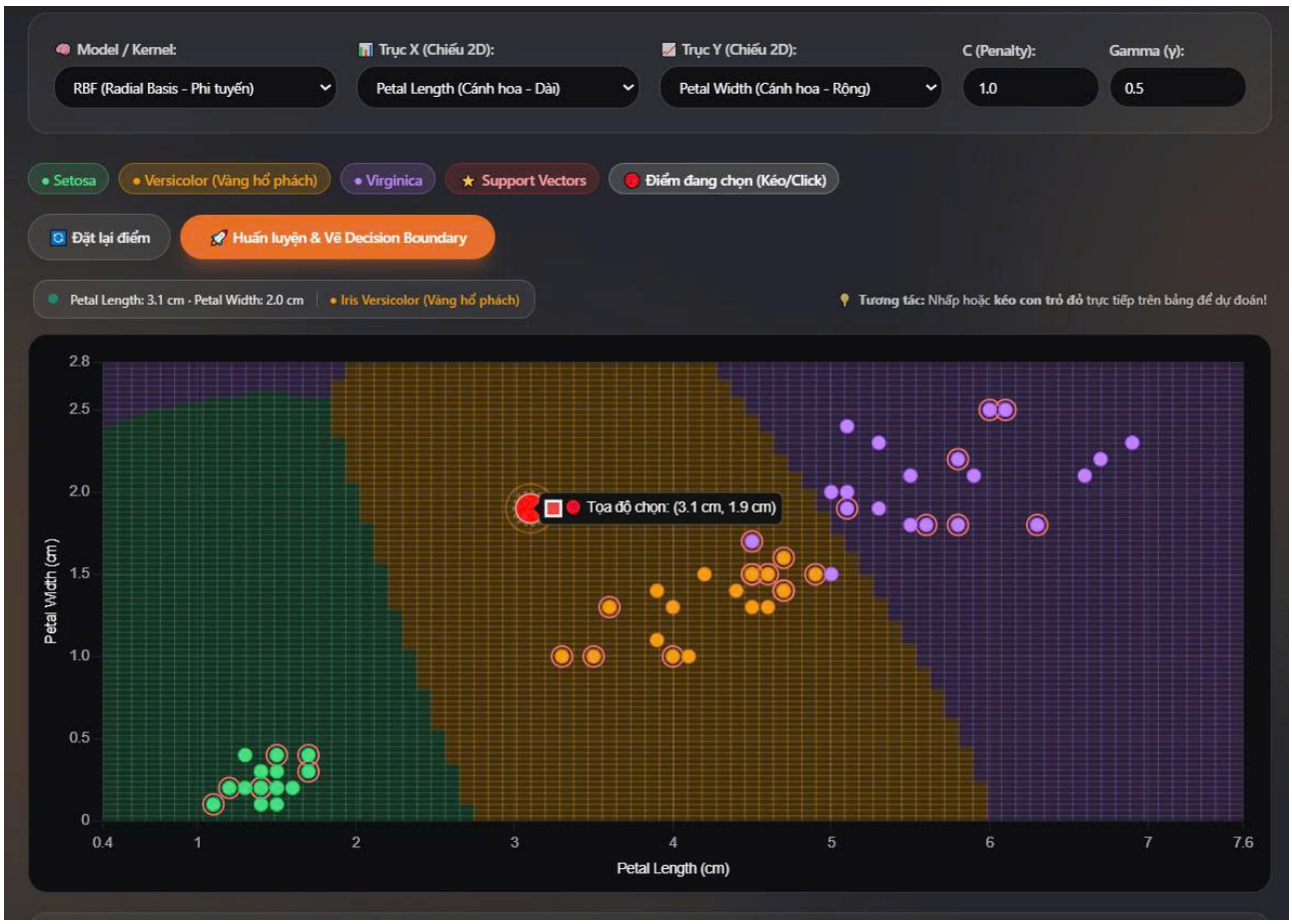
Bước 6: Cấu hình tham số mô hình. Người dùng có thể điều chỉnh các tham số quan trọng của mô hình SVM tại khu vực cấu hình RBF Kernel:

Model / Kernel: Ở đây, đang chọn RBF Kernel (Radial Basis Function - phi tuyến) để xử lý các ranh giới phân loại phức tạp.

Trục X/Y (Chiều 2D): Do ta đang trực quan 2D, ta cần chọn 2 đặc trưng chính để biểu diễn trên trục tọa độ. Ở đây, chọn 'Petal Length' cho trục X và 'Petal Width' cho trục Y, đây là hai đặc trưng có khả năng phân loại tốt nhất.

C (Penalty): Tham số điều chỉnh lỗi phân loại. Đang đặt giá trị 1.0 làm mặc định.

Gamma: Tham số quyết định mức độ ảnh hưởng của một điểm dữ liệu đơn lẻ. Đặt giá trị mặc định là 0.5.



Hình 10: Biểu đồ tương tác

Bước 7: Huấn luyện và Trực quan hóa Ranh giới quyết định. Sau khi đã cấu hình xong, bước quan trọng nhất là huấn luyện mô hình. Người dùng chỉ cần nhấn nút 'Huấn luyện & Vẽ Decision Boundary'.

Ngay lập tức, hệ thống sẽ chạy thuật toán SVM. Kết quả không chỉ là mô hình được huấn luyện mà còn là một biểu đồ trực quan hóa Decision Boundary cực kỳ rõ ràng trên nền dark mode:

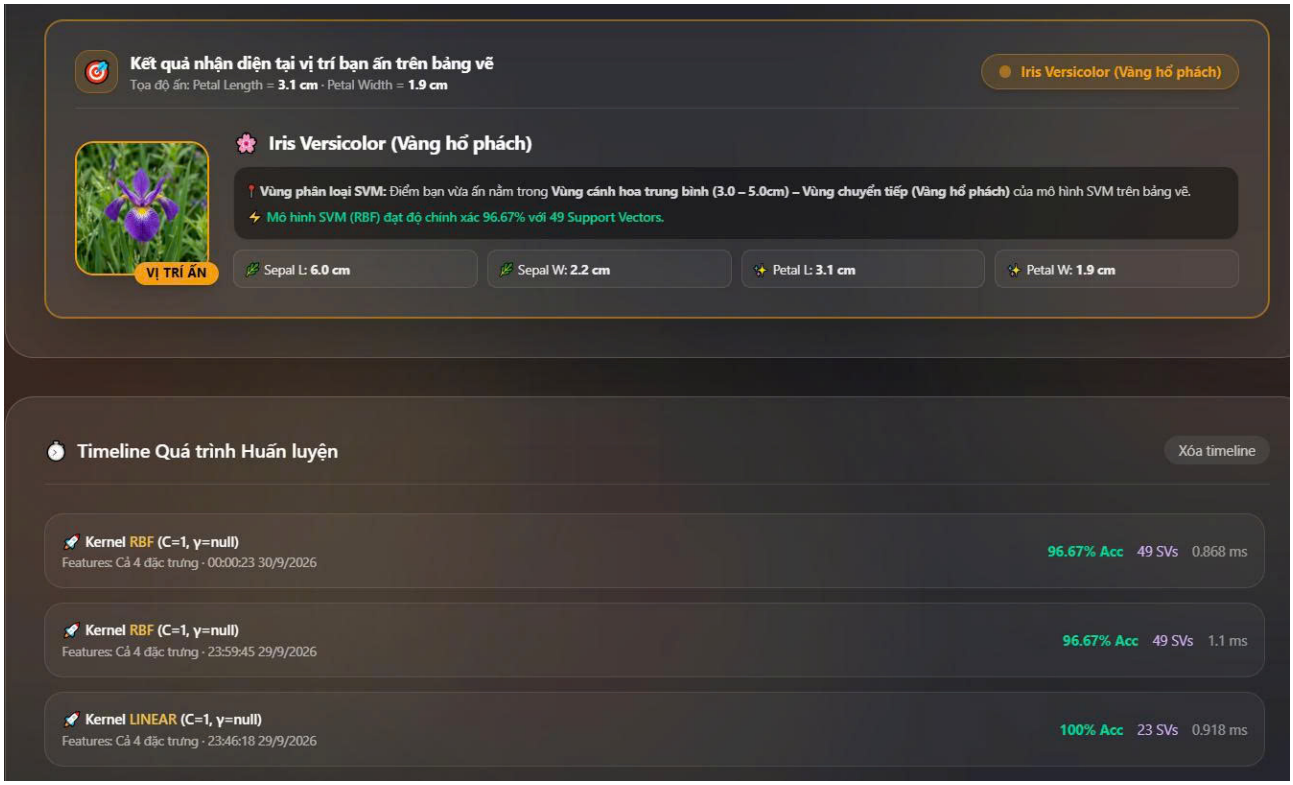
- Vùng màu xanh lá cây tương ứng với ranh giới phân loại loài Setosa
- Vùng màu cam tương ứng với ranh giới phân loại loài Versicolor
- Vùng màu tím tương ứng với ranh giới phân loại loài Virginica

Các điểm Support Vectors quan trọng được đánh dấu bằng hình ngôi sao vàng

Điểm đặc biệt và tương tác cao nhất của phân hệ này chính là khả năng dự đoán trong thời gian thực. Bạn có thể thấy một con trỏ đỏ trên biểu đồ, được gọi là 'Điểm đang chọn'.

Người dùng có thể nhấp chuột hoặc kéo trực tiếp con trỏ này trên bảng. Khi con trỏ di chuyển, giá trị tọa độ (Petal Length và Petal Width) sẽ thay đổi tương ứng. Tại vị trí hiện tại (Petal Length: 3.1 cm, Petal Width: 1.9 cm), hệ thống ngay lập tức dự đoán điểm này thuộc loài 'Iris Versicolor' và hiển thị kết quả này một cách rõ ràng dưới bảng điều khiển.

Chức năng này giúp người dùng không chỉ học về lý thuyết SVM mà còn hiểu rõ *cơ chế* của ranh giới quyết định và cách mô hình *suy luận* cho một dữ liệu mới



Hình 11: Kết quả biểu đồ tương tác

7.1.2. Chức năng 2 – Đoán hoa

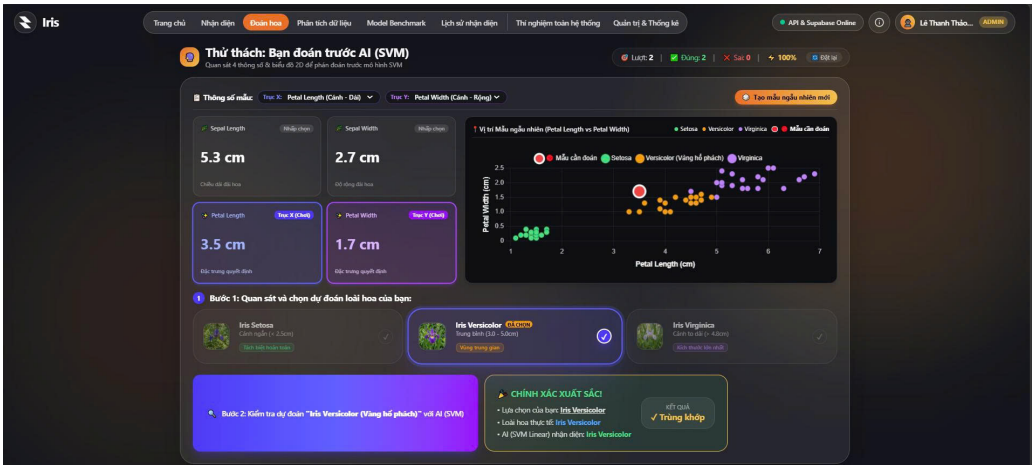
Phân hệ "Bạn đoán trước AI" được thiết kế theo hướng tương tác. Hệ thống tạo một mẫu với bốn đặc trưng, sau đó yêu cầu người dùng chọn trước một trong ba loài. Khi người dùng xác nhận, hệ thống thực hiện dự đoán và so sánh với lựa chọn.

Luồng sử dụng:

1. Mở "Bạn đoán trước AI".
2. Quan sát bốn thông số được hệ thống tạo.
3. Bước 1: Quan sát biểu đồ và chọn: Setosa, Versicolor hoặc Virginica.

4. Bước 2: Kiểm tra kết quả với AI (SVM).
5. Xem kết quả của SVM và trạng thái đúng/sai.
6. Nếu muốn, chuyển sang lượt mới để tiếp tục luyện khả năng đọc dữ liệu.

Server cũng có endpoint tạo mẫu ngẫu nhiên với các vùng khó khác nhau, trong đó có vùng biên giữa Versicolor và Virginica.



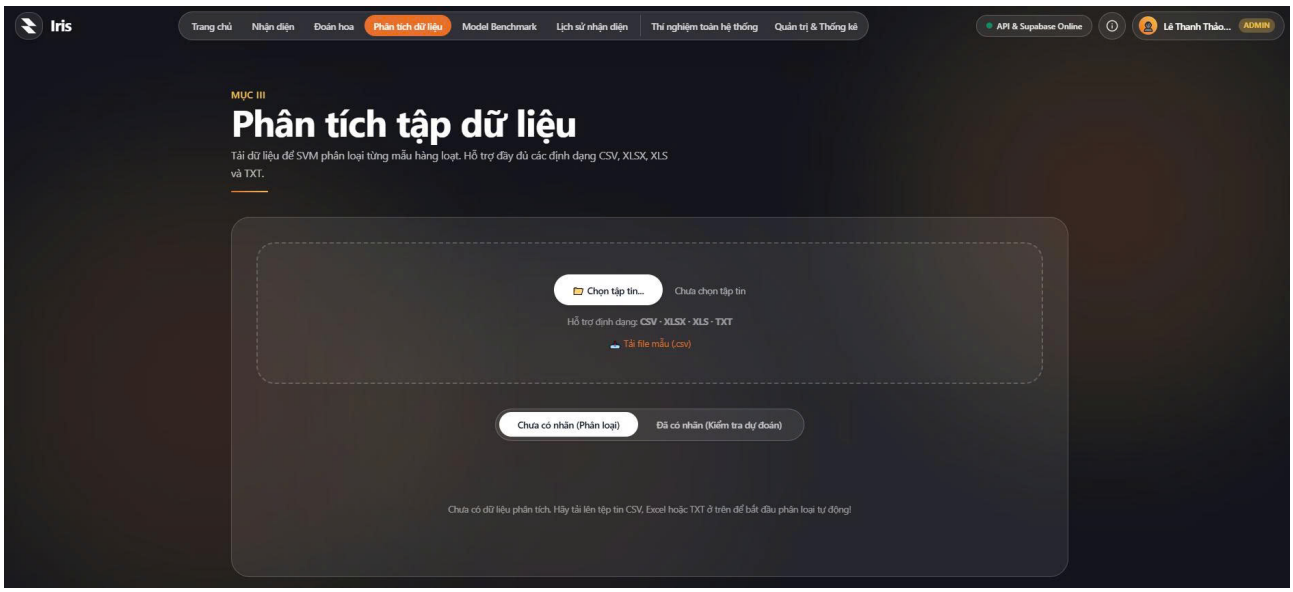
Hình 12: Thử thách "Bạn đoán trước"

7.1.3. Chức năng 3 – Phân tích dữ liệu

Phân hệ “Phân tích dữ liệu” hỗ trợ các định dạng CSV, XLSX, XLS và TXT. Người dùng có thể tải một file chứa các đặc trưng Iris để hệ thống xử lý theo từng dòng.

Quy trình sử dụng:

1. Chọn “Phân tích dữ liệu”.
2. Chuẩn bị file có bốn cột đặc trưng tương ứng Sepal Length, Sepal Width, Petal Length và Petal Width.
3. Tải file lên giao diện.
4. Chọn chế độ phù hợp nếu file có hoặc không có nhãn thật.
5. Hệ thống phân loại từng mẫu.
6. Nếu có nhãn thật, hệ thống tính số mẫu đúng và Accuracy trên các dòng hợp lệ.
7. Người dùng có thể tải kết quả phân tích dưới dạng CSV.



Hình 13: Tải file cần dự đoán lên để phân tích dữ liệu

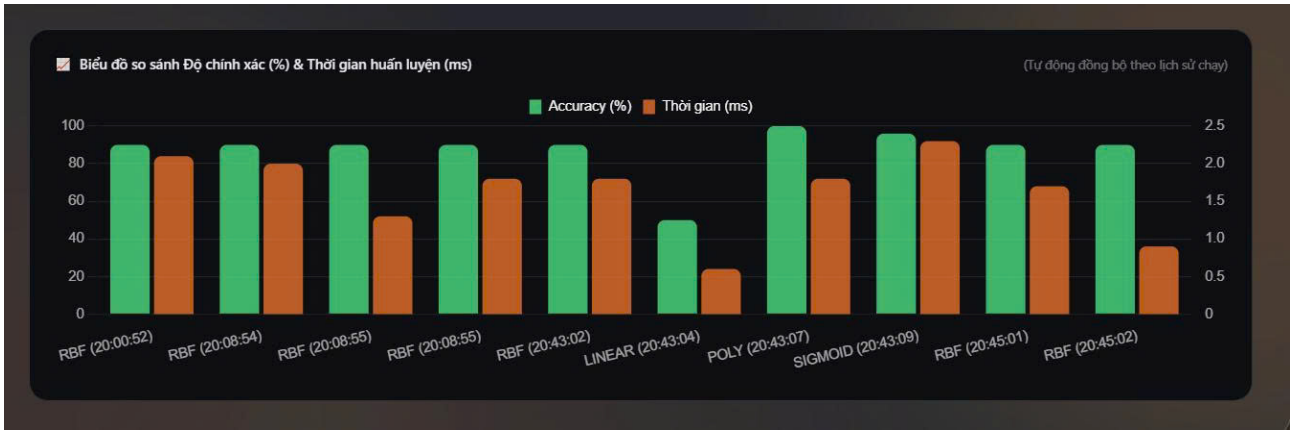
Kết quả xuất gồm các trường như đặc trưng đầu vào, nhãn thật, dự đoán SVM, độ tin cậy và trạng thái kết quả.



Hình 14: Kết quả bảng phân tích

7.1.4. Chức năng 4 – Model Benchmark

Trang Model Benchmark trình bày bảng so sánh các Kernel SVM với nhau. Mục tiêu của trang là cung cấp góc nhìn tổng quan về hiệu năng của thuật toán SVM và biểu đồ so sánh **Độ chính xác & Thời gian huấn luyện**

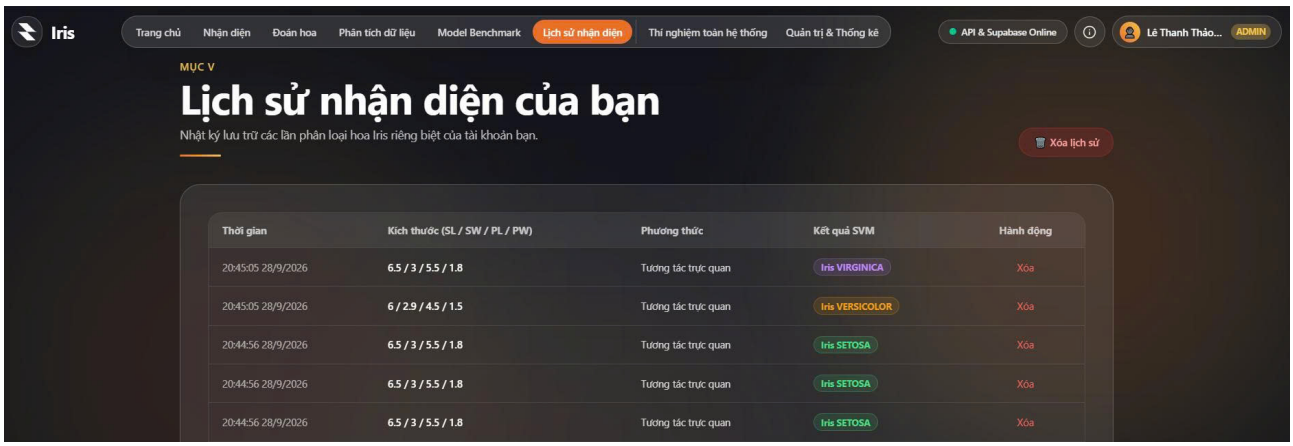


Hình 15: Biểu đồ so sánh

7.1.5. Chức năng 5 – Lịch sử nhận diện

Mỗi lần nhận diện có thể được lưu vào lịch sử cùng thời gian, bốn thông số, kết quả, độ tin cậy và phương thức thực hiện. Người dùng có thể:

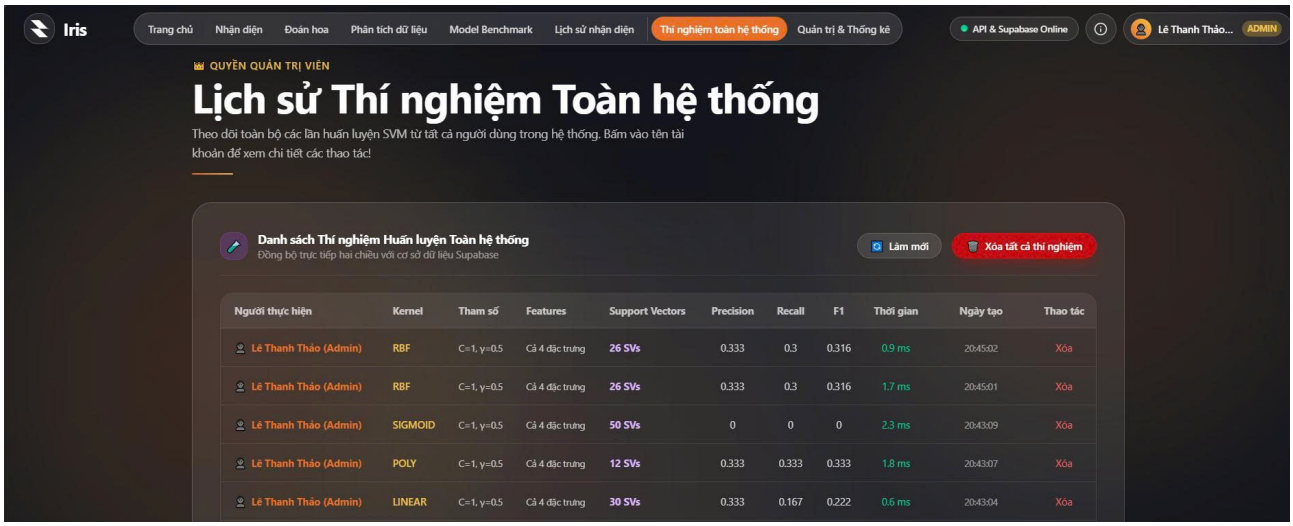
- Xem toàn bộ lịch sử.
- Xóa một bản ghi.
- Xóa toàn bộ lịch sử.



Hình 16: Lịch sử

7.1.6. Chức năng 6 – Lịch sử thí nghiệm toàn hệ thống (Dành cho quyền quản trị viên)

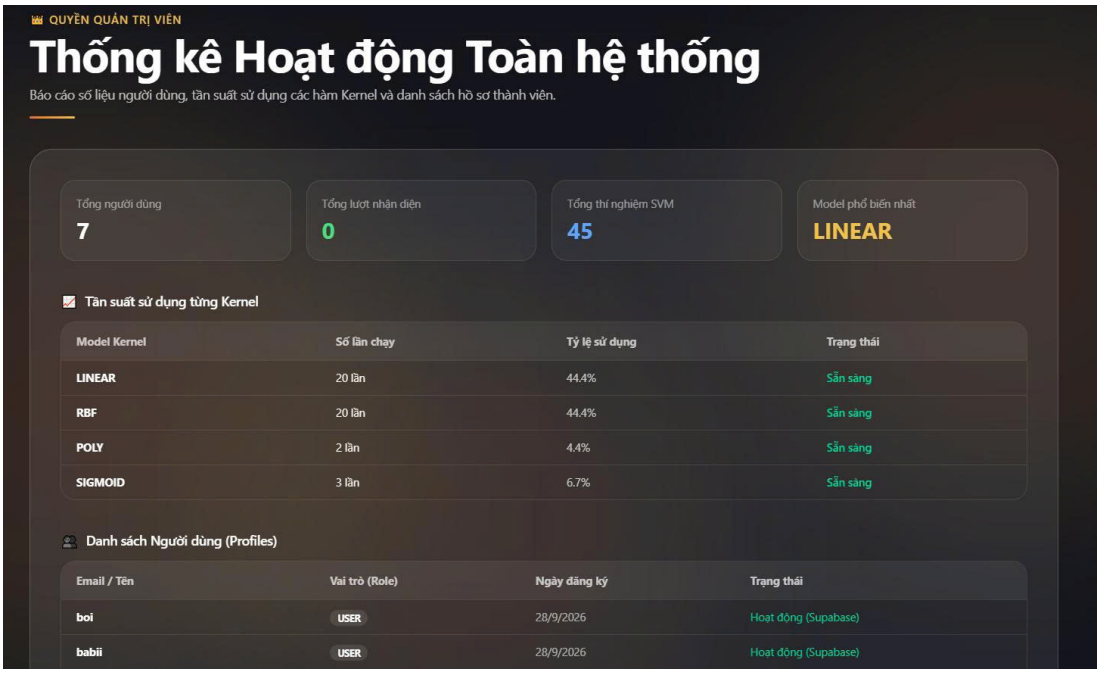
Theo dõi toàn bộ các lần huấn luyện SVM từ tất cả người dùng trong hệ thống. Bấm vào tên tài khoản để xem chi tiết các thao tác.



Hình 17: Lịch sử thí nghiệm toàn hệ thống

7.1.7. Chức năng 7 – Thống kê hoạt động toàn hệ thống (Dành cho quyền quản trị viên)

Báo cáo số liệu người dùng, tần suất sử dụng các hàm Kernel và danh sách hồ sơ thành viên.



Hình 18: Thống kê toàn hệ thống

7.2. Quy trình sử dụng đề xuất cho người mới

Đối với người dùng lần đầu, có thể thực hiện theo trình tự:

1. Dùng “Bạn đoán trước” để kiểm tra trực giác trước.
2. Vào “Nhận diện” và thử mẫu mặc định.
3. Thay đổi Petal Length và Petal Width để quan sát sự thay đổi dự đoán.
4. Vào “Huấn luyện” để hiểu phân bố ba lớp.
5. Vào “Trực quan hóa biểu đồ” và thay đổi Kernel.
6. Quan sát Accuracy và Confusion Matrix.
7. Dùng “Phân tích dữ liệu” nếu muốn xử lý nhiều mẫu từ file.
8. Vào “Model Benchmark” để so sánh thời gian chạy và độ chính xác của các Kernel.
9. Xem “Lịch sử” để kiểm tra các lần nhận diện trước đó.

KẾT LUẬN

Đề tài đã xây dựng được một hệ thống tương đối hoàn chỉnh cho bài toán phân loại hoa Iris bằng Support Vector Machine, từ lý thuyết thuật toán đến triển khai ứng dụng web. Về mặt học máy, hệ thống sử dụng bốn Kernel Linear, RBF, Polynomial và Sigmoid, cho phép quan sát rõ ảnh hưởng của hàm Kernel đến ranh giới quyết định và chất lượng phân loại.

Trên phép chia dữ liệu gồm 120 mẫu huấn luyện và 30 mẫu kiểm thử, Kernel Linear đạt Accuracy 100%, RBF và Polynomial đạt 96.67%, trong khi Sigmoid đạt 10%. Những kết quả này cho thấy cùng một thuật toán SVM nhưng lựa chọn Kernel và cấu hình tham số có thể tạo ra sự khác biệt rất lớn về kết quả. Việc sử dụng Confusion Matrix, Precision, Recall, F1-score, số Support Vector và thời gian thực thi giúp đánh giá mô hình từ nhiều góc độ thay vì chỉ dựa vào Accuracy.

Về mặt kỹ thuật phần mềm, dự án đã kết nối được nhiều thành phần: Python/scikit-learn đảm nhiệm huấn luyện, joblib đóng gói mô hình, FastAPI cung cấp dịch vụ dự đoán, React/Vite xây dựng giao diện, Express hỗ trợ lớp server frontend và Render phục vụ triển khai trực tuyến. Giao diện không chỉ cung cấp chức năng dự đoán mà còn có trực quan hóa dữ liệu, Playground, thí nghiệm, benchmark, phân tích file và lịch sử, qua đó biến một mô hình học máy thành một sản phẩm có khả năng tương tác.

Bên cạnh các kết quả đạt được, báo cáo cũng chỉ ra một số điểm cần hoàn thiện trước khi đưa hệ thống vào môi trường production, đặc biệt là chuẩn hóa dữ liệu, kiểm định chéo, tối ưu siêu tham số, đồng nhất số liệu benchmark và tăng cường bảo mật cho hệ thống tài khoản và cơ sở dữ liệu. Đây cũng là những hướng phát triển tự nhiên nếu tiếp tục mở rộng đề tài.

Nhìn chung, dự án đáp ứng được mục tiêu chính là minh họa đầy đủ vòng đời của một bài toán học máy: **dữ liệu** $\rightarrow$ **mô hình** $\rightarrow$ **đánh giá** $\rightarrow$ **API** $\rightarrow$ **giao diện** $\rightarrow$ **triển khai trực tuyến** $\rightarrow$ **vận hành**. Đây là nền tảng phù hợp để tiếp tục nghiên cứu các mô hình phân loại khác và xây dựng các hệ thống AI có khả năng phục vụ người dùng thực tế.

PHỤ LỤC

Các thành phần quan trọng của repository được tổng hợp như sau:

Bảng 8: Danh mục các tệp và thư mục chính trong dự án

Tệp / thư mục	**Vai trò**
`train.py`	Huấn luyện bốn mô hình SVM và xuất metrics.
`app.py`	FastAPI phục vụ bốn mô hình đã lưu.
`svm_linear.pkl`	Mô hình SVM Linear.
`svm_rbf.pkl`	Mô hình SVM RBF.
`svm_poly.pkl`	Mô hình SVM Polynomial.
`svm_sigmoid.pkl`	Mô hình SVM Sigmoid.
`metrics.json`	Metrics thực nghiệm của bốn Kernel.
`weights.json`	Metrics mở rộng và metadata của mô hình.
`src/App.tsx`	Điều phối giao diện và chuyển đổi các phân hệ.
`src/components/`	Các trang/chức năng của frontend.
`src/utils/api.ts`	Gọi API dự đoán từ frontend và fallback cục bộ.
`src/utils/svmModel.ts`	Thành phần SVM cục bộ cho frontend.
`src/data/irisData.ts`	Dữ liệu Iris dùng cho trực quan hóa và mô hình phía client.
`server.ts`	Express server kết hợp Vite và endpoint phụ trợ.
`render.yaml`	Cấu hình Web Service trên Render.
`package.json`	Dependency và script của frontend/server.
`supabase_schema.sql`	Schema cơ sở dữ liệu Supabase.

TÀI LIỆU THAM KHẢO

Tài liệu

[1] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[2] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, vol. 7, pp. 179–188, 1936.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] FastAPI, *FastAPI Documentation*, tài liệu tham khảo về xây dựng API bằng Python và Pydantic.

[6] React, *React Documentation*, tài liệu tham khảo về xây dựng giao diện người dùng.

[7] Vite, *Vite Documentation*, tài liệu tham khảo về công cụ build frontend.

[8] Render, *Render Documentation*, tài liệu tham khảo về triển khai Web Service.